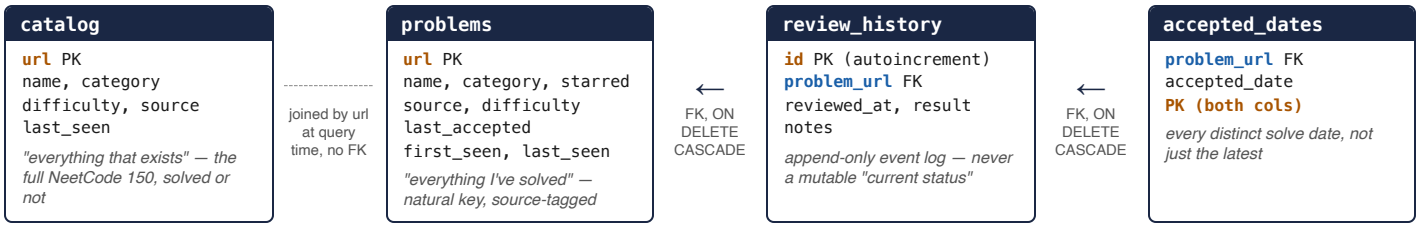
Problem Tracker — Segment 3: Data Model

3 of 8 · 8 min

The architecture · pairs with `interview-cheatsheet.pdf`'s retired content, now split across this and segments 4 & 6

Opening line: "Most of the interesting decisions in this project aren't in the UI — they're in how I chose to represent 'a problem,' 'an attempt at a problem,' and 'a problem that exists but I haven't touched yet' as three separate concepts."

The Schema at a Glance — sketch or screen-share this



Before You Start

- Sketch the four boxes and three relationships from memory — the shape matters more than exact column names.
- Headline numbers: **354 problems tracked** (239 LeetCode / 115 NeetCode), two tables enforcing a real foreign key back to `problems`.
- Two topics are deliberately *not* here even though they're schema-adjacent — the two-DB migration story and the cross-source reconciliation both get their own dedicated segments right after this one.

The 8-Minute Run of Show

0:00–0:30	Opening line (above)
0:30–2:15	Walk the four tables + relationships
2:15–4:00	Key design decisions, and why
4:00–7:00	Extending the schema live, twice
7:00–8:00	Scaling note + transition

Section-by-Section Script

Walk the tables (0:30–2:15)

- *"catalog is every problem that exists. problems is every problem I've actually solved — a strict subset. review_history is every attempt at a solved problem. accepted_dates is every distinct date it was actually accepted on the platform, independent of whether I ever reviewed it in this app."*
- *"Two of these are real foreign keys — review_history and accepted_dates both reference problems.url with ON DELETE CASCADE. catalog is the odd one out, joined by url at query time on purpose."*

Key design decisions (2:15–4:00)

- `url` **as the primary key**, not a surrogate id — *"the scraper's own natural identity for a row, so an upsert is just INSERT ... ON CONFLICT(url) DO UPDATE."*
- **Append-only, never mutable** — *"review_history and accepted_dates both derive aggregates via SQL — COUNT, MAX(id) GROUP BY — so there's no denormalized counter that can drift out of sync with the log."*
- `source` **as a discriminator**, not a table-per-source — *"a third source could slot in without a schema change."*

Extending the schema live, twice (4:00–7:00)

- **catalog:** *"I added a recommendation feature that needed the app to know about problems that exist but were never solved. The tempting shortcut was a status column on problems — I didn't do that, because it would conflate 'have I attempted this' with 'is this a real, catalogued problem' on the same row. A separate table keeps both facts independently true."*
- **accepted_dates:** *"Same call, days later, for a different reason. Both scrapers already had every accepted-submission date in hand and threw all but the latest away. A single column can't hold a variable-length history without becoming a delimited string a query can't reason about — a child table kept 'how many times has this been solved' an honest COUNT DISTINCT instead of string-parsing."*

Scaling + transition (7:00–8:00)

- *"Single-user, ~350 rows, SQLite is the right call. If this became multi-user, the real wall is SQLite's one-writer-at-a-time model — I'd move to Postgres and split problems into a shared catalog plus a per-user table."*
- *"Two things I've deliberately left out of this segment — a real mistake I made and fixed in this schema, and how I reconcile NeetCode against LeetCode — are coming up next."*

If they ask "why not a status column / a delimited string?" — cramming a second concept onto an existing column makes its meaning ambiguous by inspection, and a delimited string can't be queried, counted, or indexed — every read has to parse it back apart first. A real table keeps each fact independently true and queryable.